

Tic Tac Toe – Coursework Report

K.Mamedaitis EF25/1

1. Introduction

What is this application?

This is a terminal-based **Tic Tac Toe** game. Two players take turns placing X or O on a 3×3 grid. The first to get three in a row wins. If the grid fills up with no winner, it is a draw.

The game has two modes:

- **Human vs Human** – two people play on the same keyboard
- **Human vs AI** – one person plays against the computer

Every result is saved to a results.csv file automatically.

How to use the program

1. Choose mode 1 or 2
2. Players take turns entering a number from **1 to 9**:

```
1 | 2 | 3
---+---+---
4 | 5 | 6
---+---+---
7 | 8 | 9
```

3. The game announces a winner or a draw
4. You can choose to play again after each game
5. When you quit, all results from the session are shown

2. Body / Analysis

Encapsulation

The Board class keeps its list of cells hidden using double underscores (`__cells`). Nothing outside the class can read or change it directly — you must use the class's own methods.

python

```
class Board:
    def __init__(self):
        self.__cells = [" "] * 9 # hidden from outside

    def place(self, pos, symbol):
        self.__cells[pos] = symbol

    def is_empty(self, pos):
        return self.__cells[pos] == " "
```

Abstraction

Player is an abstract class. It forces every subclass to implement `make_move()`, but does not say how. This hides the details of *how* a move is made from the rest of the program.

python

```
from abc import ABC, abstractmethod

class Player(ABC):
    @abstractmethod
    def make_move(self, board):
        pass # subclasses must fill this in
```

Inheritance

HumanPlayer and AIPlayer both inherit from Player. They automatically get the name and symbol properties and only need to write their own `make_move()`.

python

```
class HumanPlayer(Player): # inherits from Player
    def make_move(self, board):
        pos = int(input("Pick 1-9: ")) - 1
        return pos

class AIPlayer(Player): # also inherits from Player
    def make_move(self, board):
        for i in range(9):
            if board.is_empty(i):
                return i
```

Polymorphism

In Game, the current player is stored as a Player object. The game loop calls `player.make_move(board)` without knowing or caring whether it is a human or AI — both respond to the same method, but do different things.

python

```
player = self.__players[turn % 2] #
pos = player.make_move(self.__board)
```

Design Pattern: Singleton

The ResultSaver class uses the **Singleton** pattern. This means only one instance of it can ever exist. The pattern works by overriding `__new__` — on the first call it creates the instance, and on every call after that it returns the same one.

python

```
class ResultSaver:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance
```

Composition

Game creates its own Board internally. The board cannot exist without the game.

python

```
class Game:
    def __init__(self, player1, player2):
        self.__board = Board() # Board is created here, owned by Game
```

Aggregation

The players are created *outside* Game and passed in. They could exist without a Game.

python

```
self.__players = [player1, player2]
```

Reading from and Writing to File

ResultSaver saves every game result to results.csv using Python's csv module. It appends one row per game, and reads them all back when the session ends.

Writing:

python

```
def save(self, game_num, winner):
    with open(self.filename, "a", newline="") as f:
        csv.writer(f).writerow([game_num, winner])
```

Reading:

python

```
def load(self):  
    with open(self.filename, "r") as f:  
        return list(csv.DictReader(f))
```

3. Results and Summary

This project produced a working Tic Tac Toe game built with object-oriented programming. Each class has one clear job: “Board“ manages the grid, “Player“ subclasses handle input, “ResultSaver“ handles the file, and “Game“ connects everything together.

4. Resources

- [Python abc module](#)
- [Python csv module](#)
- [Python unittest framework](#)
- [PEP 8 style guide](#)
- [Singleton pattern explained](#)